

kurvst Typst API

kurvst is a Typst package backed by `kurvst.wasm`, a small Kurbo-based plugin. Its public API is path-based: construct a path dictionary, pass that path into transforms, and draw the returned path.

It exposes Bezier utilities that are awkward or unavailable in native Typst:

- constructing cubic and Hobby paths,
- trimming paths by arc length,
- generating sampled path patterns,
- fitting Kurbo offset/parallel paths,
- converting returned path geometry to CeTZ drawing commands.

Choose an import path

Kurvst is currently a bundled source package, not a Typst Universe package. A Clinnet run writes it below `build/templates/crates/kurvst/typst/`; a custom template in `build/templates/` can therefore import it with:

```
#import "crates/kurvst/typst/src/lib.typ" as kurvst
```

Repository examples live one directory beside `src/` and instead use `#import "../src/lib.typ" as kurvst`. Keep `kurvst.wasm`, `typst.toml`, and the `src/` tree together when copying the package to another Typst project.

Path geometry points are two-item tuples:

```
#let p = (1.2, -0.4)
```

Cubic segments use `start`, `control-start`, `control-end`, and `end`:

```
#let segment = (  
  start: (0, 0),  
  control-start: (1, 0.5),  
  control-end: (2, -0.5),  
  end: (3, 0),  
)
```

Use `from-cubic(segment)` once to turn that cubic dictionary into a path dictionary. Returned paths contain a curve-command wire path in `path`. Rust converts that wire path to Kurbo's `BezPath` internally, and every path-level Kurvst function accepts either a returned dictionary or the raw wire path.

```
#let path = kurvst.from-cubic(segment)  
#let trimmed = kurvst.trim(path, start-outset: 0.2, end-outset: 0.1)  
#let shifted = kurvst.parallel(trimmed, distance: 0.15)
```

Path Wire Format

A Kurvst wire path is a dictionary with one `elements` array. The elements mirror Typst's native curve commands, but points are plain numeric tuples:

```
#let path = (  
  elements: (  
    (kind: "move", start: (0, 0)),  
    (kind: "line", end: (0.6, 0.3)),  
    (kind: "quad", control: (1.0, 1.0), end: (1.4, 0.3)),  
    (  
      kind: "cubic",  
      control-start: (1.8, -0.4),  
      control-end: (2.4, 1.0),  
      end: (3.0, 0),  
    ),  
    (kind: "close", mode: "straight"),  
  ),  
)
```

The supported element kinds are:

- `move`: starts a subpath at `start`.
- `line`: adds a straight segment ending at `end`.
- `quad`: adds a quadratic segment using `control` and `end`.

- cubic: adds a cubic segment using control-start, control-end, and end.
- close: closes the current subpath. mode is optional and defaults to "straight" when emitted by Kurvst.

Prefer the path fragment helpers for new code. line, quad, and cubic include their start points, so fragments can be built independently:

```
#let custom = kurvst.path(
  kurvst.line((0, 0), (0.6, 0.3)),
  kurvst.quad((0.6, 0.3), (1.0, 1.0), (1.4, 0.3)),
  kurvst.cubic((1.4, 0.3), (1.8, -0.4), (2.4, 1.0), (3.0, 0)),
)
```

path and append flatten fragments. If an appended fragment starts where the current path ends, its leading move is skipped; if it starts elsewhere, the move begins a new subpath. The edited path can go straight back into Kurvst transforms:

```
#let base = kurvst.from-cubic(segment)
#let extended = kurvst.append(base, kurvst.line(segment.end, (3.4, 0.2)))

#let trimmed = kurvst.trim(extended, start-outset: 0.15)
#let shifted = kurvst.parallel(trimmed, distance: 0.1)

#kurvst.to-native(shifted, unit: 36pt, stroke: rgb("#1b7f4c") + 0.7pt)
```

Use elements when you need the raw command stream, points for the visited endpoints, segments for cubic segment dictionaries, to-native for native Typst drawing, and to-cetz-data or to-cetz for CeTZ.

A typical chain keeps the returned dictionaries intact until the final drawing step:

```
#let base = kurvst.hobby-through(demo-start, demo-through, demo-end)
#let trimmed = kurvst.trim(base, start-outset: 0.2, end-outset: 0.1)
#let shifted = kurvst.parallel(trimmed, distance: 0.15)

#cetz.canvas({
  kurvst.to-cetz(base, stroke: rgb("#bbbbbb") + 0.4pt)
  kurvst.to-cetz(trimmed, stroke: rgb("#d72638") + 0.6pt)
  kurvst.to-cetz(shifted, stroke: rgb("#1b7f4c") + 0.6pt)
})
```

Paths And Trimming

cubic constructs a path from one cubic Bezier. trim trims by curve distance from the start and/or end and returns another path dictionary.

```
#let path = kurvst.from-cubic(segment)
#let trimmed = kurvst.trim(path, start-outset: 0.15, end-outset: 0.1)
```

intersections(a, b) returns transverse crossings sorted by arc distance along a. Each record includes point, distance-a, distance-b, segment indices, and local segment parameters. Path-endpoint and tangential contacts are omitted, so the distances can be passed directly to path-splitting logic.

hobby-through(start, through, end) builds a smooth open curve through three points; segments returns the two cubic halves split at through. hobby-spline(points) does the same for any open point sequence with at least two points. Both return path dictionaries and can be passed directly to path-level helpers.

```
#let spline = kurvst.hobby-spline((
  (0.0, 0.0),
  (0.9, 0.8),
  (1.8, -0.3),
  (2.8, 0.4),
))
#let shifted = kurvst.parallel(spline, distance: 0.14)
#let wiggle = kurvst.pattern(
  spline,
  pattern: kurvst.wave(samples-per-period: 24),
  amplitude: 0.1,
  wavelength: 0.55,
```

```

)
#cetz.canvas({
  kurvst.to-cetz(spline, stroke: rgb("#bbbbbb") + 0.45pt)
  kurvst.to-cetz(shifted, stroke: rgb("#1b7f4c") + 0.6pt)
  kurvst.to-cetz(wiggle, stroke: rgb("#355c9a") + 0.55pt)
})

```

split-through combines Hobby spline construction with endpoint trimming and returns one path part between each consecutive point. Consumers such as Linnest can wrap it for graph-edge geometry.

Path Patterns

pattern generates a one-dimensional pattern along any path dictionary. Built-ins are regular Typst dictionaries:

```

#let wave = kurvst.wave()
#let zigzag = kurvst.zigzag()
#let coil = kurvst.coil(longitudinal-scale: 1.6)

```

String names "wave", "zigzag", and "coil" are accepted for convenience, but they are resolved in Typst before calling wasm.

Custom patterns use the same object shape:

```

#let hook = (
  kind: "points",
  name: "hook",
  interpolation: "linear",
  points: (
    (at: 0, x: 0, y: 0),
    (at: 0.25, x: 0.15, y: 1),
    (at: 0.5, x: 0, y: 0),
    (at: 0.75, x: -0.15, y: -1),
    (at: 1, x: 0, y: 0),
  ),
)

```

at is the phase position inside one wavelength, from 0 to 1. The x coordinate offsets along the path tangent and y offsets along the path normal; both are scaled by amplitude. If at is omitted, points are spaced evenly. Use interpolation: "linear" for corners and "smooth" for a spline through sampled points.

kurvst.coil defaults to samples-per-period: 16, longitudinal-scale: 1.25, fit-length: none, amplitude: 0.1, and wavelength: 1.0. With fit-length: none, the periodic coil is unchanged; amplitude and wavelength are used only for fitting.

Set fit-length: L to return an ordinary normalized point-pattern dictionary spanning $P = \max(1, \text{round}(L / \text{wavelength})) - 0.5$ coil periods, using the requested wavelength. It retains full amplitude with inward endpoint phases, exact zero offsets at both ends, and endpoint-ramp: false: no taper, straight stubs, or connectors.

Longitudinal fitting depends on amplitude, so apply the dictionary once over a carrier of length L with the same amplitude, a wavelength of L, all its samples, and zero phase:

```

#let base = kurvst.from-cubic(segment)
#let L = kurvst.length(base)
#let A = 0.12
#let coil = kurvst.coil(fit-length: L, amplitude: A, wavelength: 0.55)
#let fitted = kurvst.pattern(
  base,
  pattern: coil,
  amplitude: A,
  wavelength: L,
  samples-per-period: coil.points.len() - 1,
  phase: 0,
)

```

Fitting requires positive, finite fit-length and wavelength, finite amplitude, and finite, nonnegative longitudinal-scale. Fitting happens in Typst; kurvst.pattern has no natural-endpoints argument. As for

other point patterns, curved carriers use local tangent/normal offsets, not evaluation at a corrected arc distance. `endpoint-slope` has no effect with `endpoint-ramp: false`.

Patterns with `endpoint-ramp: true`, including periodic coils, taper amplitude over 75% of a wavelength at each anchored endpoint, capped at half the path length for short paths. This is useful for coils, whose longitudinal offset would otherwise put the first and last visible points inside the turn near nodes. The longitudinal offset uses the square of the taper envelope, letting the coil open sideways before it starts doubling back.

On `kurvst.pattern`, `endpoint-slope: 0` keeps the taper tangential to the base path. Values from 0 to 3 control the envelope's initial slope; try 1 for an angled finish with coil phase: `calc.pi / 2`. This keeps the endpoint attached and still flattens the envelope into the full-sized coil. It is not an angle: the direction also depends on amplitude, wavelength, and the lateral offset at the endpoint phase. Unanchored endpoints and patterns without `endpoint-ramp` are unaffected.

```
#let base = kurvst.from-cubic(segment)
#let path = kurvst.pattern(
  base,
  pattern: hook,
  amplitude: 0.15,
  wavelength: 0.7,
)
#native-scene({
  native-cubics(kurvst.segments(base), stroke: rgb("#c8c8c8") + 0.45pt)
  native-polyline(kurvst.points(path), stroke: rgb("#1b7f4c") + 0.8pt)
})
```

Parallel Paths

`parallel` uses Kurbo's offset curve fitter to produce a path at a fixed normal distance from the source path. Positive distances follow the left normal of the path direction; negative distances follow the right normal. `start-outset` and `end-outset` trim the fitted path by arc length after the offset is computed.

```
#let base = kurvst.from-cubic(segment)
#let left = kurvst.parallel(base, distance: 0.18, start-outset: 0.35, end-outset: 0.35)
#let right = kurvst.parallel(base, distance: -0.18)
#native-scene({
  native-cubics(kurvst.segments(base), stroke: rgb("#c8c8c8") + 0.45pt)
  native-cubics(kurvst.segments(left), stroke: rgb("#1b7f4c") + 0.8pt)
  native-cubics(kurvst.segments(right), stroke: rgb("#355c9a") + 0.8pt)
})
```

Path Layers

`layer` is a convenience wrapper for drawing derived visible paths. It combines side-aware offsetting, endpoint outsets, and shortening into one path-in/path-out operation. Offsetting happens first: `length` is a fixed visible arc length, `ratio` is a fraction of the full offset path length before trimming, and `resolve-length` decides how to combine them. The default "min" keeps whichever limit is shorter. `start-outset`, `end-outset`, and `shift` are arc distances on the offset path. Positive `shift` moves the shortened interval toward the path end; it is clamped to preserve the endpoint outsets.

```
#let base = kurvst.hobby-spline((
  (0.0, 0.0),
  (0.9, 0.8),
  (1.8, -0.3),
  (2.8, 0.4),
))
#let layer = kurvst.layer(
  base,
  offset: 0.16,
  length: 1.6,
  ratio: 0.5,
  resolve-length: "min",
  shift: 0.35,
)
#cetz.canvas({
```

```

kurvst.to-cetz(base, stroke: rgb("#bbbbbb") + 0.45pt)
kurvst.to-cetz(layer, stroke: rgb("#1b7f4c") + 0.8pt)
})

```

Use `side-point` to choose the sign of the offset from a point on the desired side of the original path, before offsetting or trimming. Drawing packages can use this to place derived layers on the same side as an edge label without knowing anything about the physics or graph style that requested the layer.

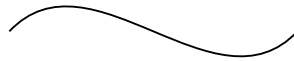
Native Drawing Primitives

Kurvst returns dictionaries and arrays. The core geometry can be drawn without CeTZ by emitting native curve content:

```

#kurvst.to-native(kurvst.from-cubic(segment), unit:
36pt, stroke: black + 0.7pt)

```



The generated reference below keeps drawing code inline. It only uses the small common helpers above for coordinate scaling, native curve construction, point markers, and fixed-size preview blocks.

CeTZ Interoperability

The CeTZ helpers are thin adapters over the same returned geometry. Use them when the surrounding document already lives in a CeTZ canvas, or when you want CeTZ path merging and styling:

```

#ceztz.canvas({
  let base = kurvst.from-cubic(segment)
  kurvst.to-cetz(base, stroke: rgb("#d72638") +
0.6pt)

  let path = kurvst.pattern(
    base,
    pattern: kurvst.coil(longitudinal-scale: 1.6),
    amplitude: 0.12,
    wavelength: 0.55,
  )
  kurvst.to-cetz(path, stroke: rgb("#355c9a") +
0.55pt)
})

```



Generated Reference

kurvst

- `append()`
- `center-outset()`
- `close()`
- `coil()`
- `cubic()`
- `cubic-point()`
- `cubic-tangent()`
- `cubic-to()`
- `elements()`
- `from-cubic()`
- `from-elements()`
- `hobby-spline()`
- `hobby-through()`
- `intersections()`
- `layer()`
- `length()`
- `line()`
- `line-segment()`

- line-to()
- move-to()
- outset-point()
- parallel()
- path()
- pattern()
- point()
- points()
- quad()
- quad-to()
- resolve-length()
- segments()
- split-through()
- to-cetz()
- to-cetz-data()
- to-native()
- trim()
- wave()
- zigzag()

Variables

- layer-defaults

append

Return a path with additional fragments or elements appended.

Parameters

```
append(
  path: dictionary,
  ..parts: any
) -> dictionary
```

path dictionary

Base Kurvst path dictionary.

..parts any

Path fragments, path elements, or element arrays to append.

center-outset

Compute the symmetric trim needed to center a shorter path layer.

Parameters

```
center-outset(
  base-length: int float,
  length: none int float,
  ratio: none int float,
  resolve-length: string function,
  start-outset: int float,
  end-outset: int float
) -> int float
```

base-length `int` or `float`

Full base path arc length.

length `none` or `int` or `float`

Fixed target visible length.

Default: `none`

ratio `none` or `int` or `float`

Relative target visible length as a fraction of base-length.

Default: `none`

resolve-length `string` or `function`

Resolution strategy for fixed and relative targets.

Default: `"min"`

start-outset `int` or `float`

Already-applied trim at the start of the path.

Default: `0`

end-outset `int` or `float`

Already-applied trim at the end of the path.

Default: `0`

close

Build a close path element.

Parameters

`close(mode: string) -> dictionary`

mode `string`

Native Typst curve close mode.

Default: `"straight"`

coil

A smooth coil path pattern.

Parameters

```
coil(  
    samples-per-period: int,  
    longitudinal-scale: int float,  
    fit-length: none int float,  
    amplitude: int float,  
    wavelength: int float  
) -> dictionary
```

samples-per-period int

Number of samples used to approximate one coil period.

Default: 16

longitudinal-scale int or float

Horizontal scale of the coil before it is mapped onto a path.

Default: 1.25

fit-length none or int or float

Fit a complete coil between inward-facing natural endpoints, without tapering. A positive path length fits half-integer periods using wavelength as nominal spacing. Apply once with pattern wavelength equal to fit-length, phase zero, and samples-per-period equal to the returned points.len() - 1.

Default: none

amplitude int or float

Amplitude used for longitudinal fitting; pass the same amplitude to pattern.

Default: 0.1

wavelength int or float

Requested coil spacing when fit-length is set.

Default: 1.0

cubic

Build a cubic path fragment.

Parameters

```
cubic(  
    start: array,  
    control-start: array,  
    control-end: array,  
    end: array  
) -> dictionary
```

start array

Start point.

control-start array

Cubic control point near start.

control-end array

Cubic control point near end.

end array

Endpoint.

cubic-point

Evaluate a cubic segment at parameter t.

Parameters

```
cubic-point(  
  segment: dictionary,  
  t: int float  
) -> array
```

segment dictionary

Segment with start, control-start, control-end, and end.

t int or float

Segment parameter in the range [0, 1].

cubic-tangent

Evaluate the tangent of a cubic segment at parameter t.

Parameters

```
cubic-tangent(  
  segment: dictionary,  
  t: int float  
) -> array
```

segment dictionary

Segment with start, control-start, control-end, and end.

t int or float

Segment parameter in the range [0, 1].

cubic-to

Build a cubic path element.

Parameters

```
cubic-to(  
  control-start: array ,  
  control-end: array ,  
  end: array  
) -> dictionary
```

control-start array

Cubic control point near the start point.

control-end array

Cubic control point near the endpoint.

end array

Cubic endpoint.

elements

Return the command elements that make up a Kurvst path.

Parameters

```
elements(path: dictionary) -> array
```

path dictionary

Kurvst path dictionary to inspect.

from-cubic

Build a path fragment from a cubic segment dictionary.

Parameters

```
from-cubic(segment: dictionary) -> dictionary
```

segment dictionary

Segment with start, control-start, control-end, and end.

from-elements

Build a path dictionary from an existing element array.

Parameters

```
from-elements(elements: array) -> dictionary
```

elements array

Array of Kurvst path elements.

hobby-spline

Construct a Hobby spline through an arbitrary point sequence.

Parameters

```
hobby-spline(  
  points: array ,  
  omega: float ,  
  accuracy: float  
) -> dictionary
```

points array

Two or more points for the open spline to pass through.

omega float

Hobby curl/tension parameter.

Default: 1.0

accuracy float

Geometry approximation accuracy passed to the Rust geometry engine.

Default: 0.001

hobby-through

Construct a cubic Hobby path through three points.

Parameters

```
hobby-through(  
  start: array ,  
  through: array ,  
  end: array ,  
  omega: float ,  
  accuracy: float  
) -> dictionary
```

start array

Start point.

through array

Intermediate point that the curve passes through.

end `array`

Endpoint.

omega `float`

Hobby curl/tension parameter.

Default: `1.0`

accuracy `float`

Geometry approximation accuracy passed to the Rust geometry engine.

Default: `0.001`

intersections

Find transverse crossings between two paths, sorted by arc distance along the first path.

Parameters

```
intersections(  
  a: dictionary ,  
  b: dictionary ,  
  accuracy: float  
) -> array
```

a `dictionary`

First Kurvst path dictionary.

b `dictionary`

Second Kurvst path dictionary.

accuracy `float`

Absolute geometry and arc-length tolerance.

Default: `0.001`

layer

Build a derived visible path layer.

Parameters

```
layer(  
  path: dictionary,  
  offset: int float,  
  length: none int float,  
  ratio: none int float,  
  resolve-length: string function,  
  shift: int float,  
  start-outset: int float,  
  end-outset: int float,  
  side-point: none array,  
  accuracy: float,  
  optimize: bool  
) -> dictionary
```

path dictionary

Base Kurvst path dictionary.

offset int or float

Signed normal offset distance.

Default: 0

length none or int or float

Fixed target visible length.

Default: none

ratio none or int or float

Relative target visible length as a fraction of the full offset path length.

Default: none

resolve-length string or function

Resolution strategy for fixed and relative targets.

Default: "min"

shift int or float

Arc-length displacement on the offset path; positive moves toward its end.

Default: 0

start-outset int or float

Arc length removed from the start.

Default: 0

end-outset `int` or `float`

Arc length removed from the end.

Default: `0`

side-point `none` or `array`

Optional point used to choose the sign of offset.

Default: `none`

accuracy `float`

Geometry approximation accuracy passed to the Rust geometry engine.

Default: `0.001`

optimize `bool`

Let Kurbo simplify/optimize fitted parallel paths.

Default: `true`

length

Compute the arc length of a path dictionary.

Parameters

```
length(  
  path: dictionary,  
  accuracy: float  
) -> int float
```

path `dictionary`

Kurvst path dictionary to measure.

accuracy `float`

Arc-length approximation accuracy passed to the Rust geometry engine.

Default: `0.001`

line

Build a straight-line path fragment.

Parameters

```
line(  
  start: array,  
  end: array  
) -> dictionary
```

start array

Start point.

end array

Endpoint.

line-segment

Build a cubic segment dictionary for a straight line.

Parameters

```
line-segment(  
  start: array ,  
  end: array  
) -> dictionary
```

start array

Start point.

end array

Endpoint.

line-to

Build a line path element.

Parameters

```
line-to(end: array ) -> dictionary
```

end array

Line endpoint.

move-to

Build a move path element.

Parameters

```
move-to(start: array ) -> dictionary
```

start array

New current point and subpath start.

outset-point

Return from moved toward toward by distance.

Parameters

```
outset-point(  
    from: array,  
    toward: array,  
    distance: int float  
) -> array
```

from array

Point to move.

toward array

Target point that defines the direction.

distance int or float

Distance to move from from toward toward.

Default: 0

parallel

Generate a parallel path for a path.

Parameters

```
parallel(  
    path: dictionary,  
    distance: int float,  
    start-outset: int float,  
    end-outset: int float,  
    accuracy: float,  
    optimize: bool  
) -> dictionary
```

path dictionary

Base Kurvst path dictionary.

distance int or float

Signed normal offset distance.

Default: 0

start-outset int or float

Arc length removed from the start of the offset path.

Default: 0

end-outset `int` or `float`

Arc length removed from the end of the offset path.

Default: `0`

accuracy `float`

Geometry approximation accuracy passed to the Rust geometry engine.

Default: `0.001`

optimize `bool`

Let Kurbo simplify/optimize the fitted path.

Default: `true`

path

Build a path dictionary from path fragments or elements.

Parameters

`path(..parts: any) -> dictionary`

..parts `any`

Path fragments, path elements, or element arrays to concatenate.

pattern

Apply a repeated path pattern to a base path.

Parameters

```
pattern(  
  path: dictionary,  
  pattern: string dictionary,  
  amplitude: int float,  
  wavelength: int float,  
  phase: int float,  
  samples-per-period: int,  
  coil-longitudinal-scale: int float,  
  anchor-start: bool,  
  anchor-end: bool,  
  endpoint-slope: int float,  
  accuracy: float  
) -> dictionary
```

path `dictionary`

Base Kurvst path dictionary.

pattern `string` or `dictionary`

Pattern dictionary or built-in pattern name.

Default: `"wave"`

amplitude `int` or `float`

Normal amplitude of the pattern in path units.

Default: `0.1`

wavelength `int` or `float`

Arc length of one pattern period.

Default: `1.0`

phase `int` or `float`

Initial phase offset in radians.

Default: `0`

samples-per-period `int`

Samples per period for string-resolved patterns and smooth point-pattern mapping.

Default: `16`

coil-longitudinal-scale `int` or `float`

Longitudinal scale used when resolving the built-in coil pattern.

Default: `1.25`

anchor-start `bool`

Force the generated path to start on the base path.

Default: `true`

anchor-end `bool`

Force the generated path to end on the base path.

Default: `true`

endpoint-slope `int` or `float`

Initial slope of an anchored taper envelope, from 0 to 3. Zero keeps tangential ends; positive values allow angled ends when the pattern's lateral offset is nonzero at its endpoint phase. This is not an angle. Ignored for unanchored ends and patterns without endpoint ramping, including fitted coils.

Default: `0`

accuracy `float`

Geometry approximation accuracy passed to the Rust geometry engine.

Default: `0.001`

point

Build a numeric point tuple.

Parameters

```
point(  
    x: int float ,  
    y: int float  
) -> array
```

x `int` or `float`

X coordinate.

y `int` or `float`

Y coordinate.

points

Return the points visited by a Kurvst path's command stream.

Parameters

```
points(path: dictionary) -> array
```

path `dictionary`

Kurvst path dictionary to inspect.

quad

Build a quadratic path fragment.

Parameters

```
quad(  
    start: array ,  
    control: array ,  
    end: array  
) -> dictionary
```

start `array`

Start point.

control array

Quadratic control point.

end array

Endpoint.

quad-to

Build a quad path element.

Parameters

```
quad-to(  
    control: array ,  
    end: array  
) -> dictionary
```

control array

Quadratic control point.

end array

Quadratic endpoint.

resolve-length

Resolve a fixed and relative visible path length.

Parameters

```
resolve-length(  
    base-length: int float ,  
    length: none int float ,  
    ratio: none int float ,  
    method: string function  
) -> none int float
```

base-length int or float

Full base path arc length.

length none or int or float

Fixed target arc length.

Default: none

ratio `none` or `int` or `float`

Relative target length as a fraction of base-length.

Default: `none`

method `string` or `function`

Resolution strategy for fixed and relative targets.

Default: `"min"`

segments

Return drawable cubic segments for any Kurvst path dictionary.

Parameters

`segments`(path: `dictionary`) -> `array`

path `dictionary`

Kurvst path dictionary to convert.

split-through

Split a path through a point sequence into per-span paths.

Parameters

```
split-through(  
  points: array,  
  omega: float,  
  start-outset: int float,  
  end-outset: int float,  
  accuracy: float  
) -> dictionary
```

points `array`

Two or more points for the curve to pass through.

omega `float`

Hobby curl/tension parameter.

Default: `1.0`

start-outset `int` or `float`

Arc length removed from the first span start.

Default: `0`

end-outset `int` or `float`

Arc length removed from the last span end.

Default: `0`

accuracy `float`

Geometry approximation accuracy passed to the Rust geometry engine.

Default: `0.001`

to-cetz

Draw a path dictionary through CeTZ.

Parameters

```
to-cetz(  
  path: dictionary,  
  unit: int float length ratio,  
  ..style: any  
) -> content
```

path `dictionary`

Kurvst path dictionary to draw.

unit `int` or `float` or `length` or `ratio`

Coordinate multiplier for emitted CeTZ points.

Default: `1`

..style `any`

CeTZ draw style arguments forwarded to merge-path.

to-cetz-data

Emit a Kurvst path as CeTZ path data.

Parameters

```
to-cetz-data(  
  path: dictionary,  
  unit: int float length ratio  
) -> array
```

path `dictionary`

Kurvst path dictionary to convert.

unit `int` or `float` or `length` or `ratio`

Coordinate multiplier for emitted CeTZ points.

Default: `1`

to-native

Emit a Kurvst path as native Typst curve content.

Parameters

```
to-native(  
  path: dictionary ,  
  unit: int float length ratio ,  
  ..style: any  
) -> content
```

path `dictionary`

Kurvst path dictionary to emit.

unit `int` or `float` or `length` or `ratio`

Coordinate multiplier for emitted Typst curve points.

Default: `1`

..style `any`

Native curve style arguments.

trim

Trim a path by arc length from each end.

Parameters

```
trim(  
  path: dictionary ,  
  start-outset: int float ,  
  end-outset: int float ,  
  accuracy: float  
) -> dictionary
```

path `dictionary`

Kurvst path dictionary to trim.

start-outset `int` or `float`

Arc length removed from the start.

Default: `0`

end-outset `int` or `float`

Arc length removed from the end.

Default: `0`

accuracy `float`

Arc-length approximation accuracy passed to the Rust geometry engine.

Default: `0.001`

wave

A smooth sinusoidal path pattern.

Parameters

`wave`(samples-per-period: `int`) -> `dictionary`

samples-per-period `int`

Number of samples used to approximate one wave period.

Default: `16`

zigzag

A straight-segment triangular path pattern.

Parameters

`zigzag`() -> `dictionary`

layer-defaults `dictionary`

Default geometry options for derived path layers.